

Introduction

Every so often a problem shows up in an interview that looks like a classic pattern on the surface but hides a trap underneath. LeetCode 2904, Shortest and Lexicographically Smallest Beautiful Substring, is exactly that kind of problem. It smells like the well-known "minimum window substring" family, and most engineers reach for a two-pointer sliding window on instinct. That instinct is half right and half dangerous, and understanding exactly where it breaks down is what makes this problem worth studying.

Interviewers like this problem because it tests three things at once: whether you can correctly define what "shortest" means when there are ties, whether you understand why a greedy two-pointer shrink can silently discard a better answer, and whether you can reason precisely about monotonic counts. It's a small problem with a big lesson about when a "smells like this pattern" instinct needs to be checked against the actual requirements.

Problem Overview

You're given a binary string made only of `0` s and `1` s, along with an integer `k` . A substring is called **beautiful** if it contains exactly `k` ones — not more, not fewer.

Your task has two parts:

1. Find the length of the shortest beautiful substring anywhere in the string.
2. Among all beautiful substrings of that shortest length, return the one that comes first alphabetically.

If no substring in the entire string ever contains exactly `k` ones, return an empty string.

The key phrase is **exactly** `k` ones. A window with more ones than `k` doesn't count, and a window with fewer doesn't count either.

Example

Take the string `100011001` with `k = 3` .

- Starting at index 0, the window needs to grow until it collects three ones. That happens at index 5, giving the substring `100011` , a window of length 6.

- Starting at index 4, three ones show up by index 8, giving `11001`, a window of length 5.

Five turns out to be the shortest length achievable anywhere in this string. Now we check: is there more than one beautiful substring of length 5? In this case `11001` is the only one, so it's the final answer.

If there had been two different length-5 windows, we'd compare them character by character and keep the smaller one — the same way you'd alphabetize two words.

Intuition

The natural first instinct when you see "substring containing exactly k of something" is to reach for a two-pointer sliding window: expand the right pointer until you hit k ones, then shrink from the left to find the minimal window, sliding across the whole string in one pass.

That pattern is excellent at finding *one* minimal window. It is not built to compare every minimal window against every other one lexicographically. A shrinking window commits to whichever minimal window it finds first at a given position and moves on — it has no mechanism for reconsidering an equally short but lexicographically smaller window that appears later in the string.

So the real insight here is narrower and more useful: for *any* fixed starting index, the first moment the running count of ones hits exactly k is guaranteed to be the shortest beautiful substring starting at that index. Why? Because ones only accumulate as you move right — the count never decreases. Once you've hit k, every additional character can only make the window longer, never shorter. That means you never need to keep growing past the first hit for a given start.

This gives you a clean plan: try every starting index, grow the window until you hit exactly k ones (or run out of string), record what you found, and compare it against the best answer seen so far — first by length, then by lexicographic order if lengths tie.

Brute Force Solution

Idea: For every starting index `i`, grow a window forward, counting ones as you go. Stop the instant the count reaches `k`. Compare that window against the best one found so far.

Algorithm: 1. Initialize `best = ""`. 2. For each starting index `i` from `0` to `n-1` :- Reset `ones_count = 0` . - Walk `j` from `i` forward, incrementing `ones_count` whenever `s[j] == '1'` . - The moment `ones_count == k`, compare the substring `s[i:j+1]` against `best` : update `best` if

it's shorter, or the same length but alphabetically smaller. Then break out of the inner loop. 3. Return `best`.

Advantages: Simple to reason about, easy to prove correct, no edge-case juggling around pointer shrinking.

Disadvantages: Quadratic time — for large strings this would be too slow, though the constraints here make it a non-issue.

Time complexity: $O(n^2)$ — every starting index can grow its window up to n steps. **Space complexity:** $O(n)$ — for storing the current best substring.

Optimal Solution

Given the constraint that the string is at most 100 characters, $O(n^2)$ is already fast enough — there's no need to reach for anything cleverer for this specific problem. But it's worth walking through *why* the double loop with an early break is not a shortcut being taken for convenience — it's structurally necessary.

Start index First index count hits k Window Length

0	5	100011	6
4	8	11001	5

The table shows exactly why you can't just take the first shortest window you stumble across — you have to check *every* starting index's best candidate and then compare across them, because a shorter or equally-short but smaller window can appear starting from any position, not just the first one your scan reaches.

The proof that the break is safe: once `ones_count` hits `k` at some index `j` for a fixed start `i`, every subsequent character only extends the window rightward. Since we've already established the window ending at `j` is the shortest one possible from `i` (extending it can never shrink it back down), there is nothing to gain by continuing the inner loop. Breaking early doesn't cut corners — it eliminates work that provably cannot produce a better answer for that starting index.

Python Code

```
def shortest_beautiful_substring(s: str, k: int) -> str:
    best = ""
    n = len(s)

    for i in range(n):
```

```

ones_count = 0
for i in range(n):
    for j in range(i, n):
        if s[j] == '1':
            ones_count += 1
        if ones_count == k:
            candidate = s[i:j + 1]
            if not best or len(candidate) < len(best) or (
                len(candidate) == len(best) and candidate < best
            ):
                best = candidate
            break # further growth from this start can only get longer

return best

```

Code Walkthrough

- `best = ""` tracks the best beautiful substring found across all starting points, initialized to empty in case no valid substring exists.
- The outer loop `for i in range(n)` tries every possible starting index.
- `ones_count = 0` resets the running count for each new starting index.
- The inner loop `for j in range(i, n)` grows the window one character at a time from `i`.
- `if s[j] == '1': ones_count += 1` is the only bookkeeping needed — the count is monotonic by construction.
- `if ones_count == k:` fires exactly once per starting index — the first time the count reaches `k`.
- The comparison `len(candidate) < len(best) or (len(candidate) == len(best) and candidate < best)` enforces the two-tier rule: shorter always wins, and only equal lengths fall back to string comparison. `candidate < best` uses Python's native lexicographic string comparison.
- `break` stops the inner loop immediately after recording the candidate, since nothing useful can come from continuing.

Dry Run

String: `100011001`, `k = 3`

- **i = 0:** ones accumulate at indices 0, 4, 5 → count hits 3 at `j = 5`. Candidate: `s[0:6] = "100011"`, length 6. `best` is empty, so `best = "100011"`.
- **i = 1:** ones at 4, 5, count hits 3 at `j = 8` (index 1 itself is `'0'`). Candidate: `s[1:9] = "00011001"` ... let's recheck: actually count reaches 3 at index 8. Candidate length 8 — longer than

current best, skipped.

- **i = 4:** starting from index 4 ('0'), ones show up at indices 4? Let's index the string: 1 0 0 0 1 1 0 0 1 → indices 0-8. From **i = 4** , characters are 1,1,0,0,1 (indices 4-8). Count hits 3 at **j = 8** . Candidate: **s[4:9] = "11001"** , length 5. This beats **best** (length 6), so **best = "11001"** .
- Remaining starting indices either never reach **k = 3** ones or produce longer/no windows.

Final answer: **"11001"** — matching the expected result.

Complexity Analysis

Time: $O(n^2)$ in the worst case — n possible starting points, each potentially scanning up to n characters before hitting **k** ones or exhausting the string.

Space: $O(n)$ for storing the current best substring (slicing creates a new string each time).

These bounds are correct because the algorithm does no redundant work beyond the necessary re-scan per starting index — there's no way to avoid at least considering every starting point, since the answer could begin anywhere. With **n** capped at 100 by the problem's constraints, this runs effectively instantly.

Alternative Solutions

A true sliding window (two pointers, amortized $O(n)$) could theoretically find a minimal window quickly, but as established, it can't correctly track the lexicographically smallest one among ties without additional bookkeeping — you'd need to record and compare every minimal window it produces anyway, which erases most of the speed benefit for this problem's tiny input size. Given **n ≤ 100** , the $O(n^2)$ double loop is the pragmatic choice: it's simpler to write correctly and just as fast in practice.

Edge Cases

- **All zeros, $k \geq 1$:** No window ever reaches **k** ones. **best** stays empty, function correctly returns **""** .
- **k = 0:** Every single-character substring of '0' s (or the empty substring, depending on problem convention) would trivially satisfy zero ones — worth checking against the exact problem constraints if this is allowed.

- **k equals total ones in string:** The only valid window is the one spanning from the first to the last '1' in the string, possibly with padding zeros trimmed from consideration if a shorter one exists elsewhere.
- **String length equal to 1:** Only one starting index to check.
- **Multiple length-5 windows tied lexicographically at the first differing character:** Confirms the `candidate < best` comparison correctly does character-by-character comparison, not length-based shortcuts.

Common Mistakes

1. **Forgetting to check length before lexicographic order.** Comparing strings directly with `<` without first checking length can pick a smaller-looking but longer substring.
2. **Reaching for a shrinking two-pointer window** and assuming it'll naturally find the lexicographically smallest among ties — it won't, because it discards windows once it moves past them.
3. **Removing the `break` statement**, thinking it's just an optimization — without it, the loop keeps overwriting `candidate` with longer windows from the same start, corrupting the "shortest from this start" property.
4. **Off-by-one errors on slicing**, particularly `s[i:j]` instead of `s[i:j+1]`, which drops the character that made the count hit `k`.
5. **Not handling the no-valid-window case**, returning `None` or throwing an error instead of an empty string.

Interview Questions

- How would you modify this if the input could be very large (e.g., 10^6 characters)? Would the $O(n^2)$ approach still work?
- Can you convert this into a true $O(n)$ solution while preserving the lexicographic smallest requirement? (Hint: think about tracking a rolling window of minimal windows and comparing only where necessary.)
- What if the string contained characters other than `0` and `1`, and you needed to count occurrences of a specific character?
- How would the solution change if you needed the lexicographically *largest* beautiful substring instead?

Similar Problems

- **LeetCode 76 — Minimum Window Substring:** The classic sliding window problem this one is often confused with; useful for understanding where the shrinking window pattern applies cleanly.
- **LeetCode 3 — Longest Substring Without Repeating Characters:** Another sliding window problem, but here you're maximizing length instead of minimizing it.
- **LeetCode 209 — Minimum Size Subarray Sum:** Same "shrink once you hit a threshold" idea, but over numeric sums instead of counting a specific character.
- **LeetCode 1004 — Max Consecutive Ones III:** Reinforces counting a specific value within a sliding window, with a flip allowance twist.

Key Takeaways

The core lesson of this problem isn't the sliding window pattern itself — it's recognizing when a familiar pattern doesn't fully apply. Exactly-k-count problems tend to look like minimum-window problems, but tie-breaking with lexicographic order changes the requirements enough that a naive two-pointer shrink can silently return a wrong answer. Whenever a count only increases as a window grows, you can stop expanding the moment you hit your target — that shortcut is safe and it's the same principle that keeps this solution fast despite its quadratic upper bound.

Watch the Video

For the full walkthrough with the real trace-through example, the reasoning behind why the break statement matters, and two live quiz rounds to test your understanding before seeing the answer, check out the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to teaching real software engineering skills — not just memorized answers. Whether it's LeetCode patterns, system design, or the tools developers use daily, the goal is always the same: build the intuition first, so the code stops feeling like a trick and starts feeling obvious.

Call To Action

If this explanation helped the problem click, consider liking the video on YouTube and subscribing to the channel for more interview-focused walkthroughs. For deeper dives delivered straight to your

inbox, subscribe to the Substack newsletter. Drop a comment with your own approach or questions, and share this with a fellow developer prepping for interviews.

Quick Review

When you see 'shortest, lex-smallest string satisfying a condition', which pattern applies?

Greedy left-to-right construction: at each position pick smallest valid char, backtrack/extend only when forced by the condition.

Time complexity of scanning positions and checking the 'beautiful' condition each step?

$O(n \cdot k)$ where n is string length and k is condition-check cost (e.g. counting vowels/palindrome check per window).

Space complexity for this greedy scan approach?

$O(1)$ extra beyond output, or $O(n)$ if you store character counts/prefix info for the condition check.

Trickiest bug: string is short/lex-small locally but fails overall?

Being lex-smallest at one prefix doesn't guarantee validity later — must verify full condition holds before accepting, not just greedily commit.

What happens when no beautiful string exists for given constraints?

Return empty string; always check feasibility (e.g. required char counts vs length) before greedy construction starts.

Greedy-construction vs sliding-window: when does each apply?

Sliding window finds a substring of existing text; greedy construction builds a new string character-by-character from scratch.